Rapport technique détaillé

Fusion tardive LiDAR–caméra, freinage d'urgence automatique (AEB) validé en boucle fermée, *ADAS Validation Studio* et agent LLM de génération de scénarios d'essai

Projet personnel sous CARLA 0.9.16

Mohamed Eladeb (ENSTA Paris, 3A parcours robotique)

7 novembre 2026

Résumé

Ce document décrit l'ensemble du projet `carla-lidar-camera-fusion` : la chaîne de perception (collecte synchronisée, étiquetage automatique, détecteur YOLO fine-tuné, estimation de distance par LiDAR), le suivi de cibles par filtre de Kalman, le freinage d'urgence automatique validé en boucle fermée dans CARLA, l'interface web *ADAS Validation Studio* (API FastAPI, campagnes d'essais) et l'agent qui convertit une demande en langage naturel en scénario JSON exécutable. Pour chaque brique sont donnés : le but, la méthode, les choix de conception et leur justification, les résultats mesurés, les erreurs rencontrées et corrigées, et surtout *ce qui n'est pas démontré*. Le rapport distingue partout ce qui est **implémenté**, **testé**, **validé en simulation** ou **planifié**.

Table des matières

1 Contexte, objectifs et règles de travail **4**
1.1 Objectif du projet . . . 4
1.2 Règles de méthode (cahier des charges de conduite du projet) . . . 4
1.3 Environnement . . . 4

2 Vue d'ensemble et structure du projet **5**
2.1 Chaîne de traitement . . . 5
2.2 Arborescence . . . 5
2.3 Scripts, du plus ancien au plus récent . . . 5

3 Banc de simulation et collecte de données **6**
3.1 Capteurs et paramètres . . . 6
3.2 Étiquetage automatique . . . 6
3.3 Contrôles de qualité du jeu de données . . . 7

4 Détecteur YOLO **7**
4.1 Choix et entraînement . . . 7
4.2 Résultats sur le jeu de test (recall par distance et météo) . . . 8

5 Fusion LiDAR–caméra : distance et rejet de fausses alarmes 8
5.1 Méthode (fusion tardive) . . . 8
5.2 Comparaison à deux estimations caméra seule . . . 9
5.3 Rejet des détections sans preuve LiDAR . . . 9
5.4 Mesure latérale . . . 10

6 Suivi de cibles (filtre de Kalman) 10

7 AEB : méthode et choix de conception 10
7.1 Machine d'états . . . 10
7.2 Critère principal : la décélération requise . . . 11
7.3 Mesures de robustesse . . . 11

8 Validation en boucle fermée dans CARLA 12
8.1 Banc de test . . . 12
8.2 Résultats, Town10HD_Opt, 20 Hz, météo claire . . . 12
8.3 Effet de la fréquence de perception . . . 12
8.4 Seconde ville : Town03 . . . 13
8.5 Un cas non évitable, volontairement exclu des tableaux . . . 13

9 Robustesse aux fausses détections 13
9.1 Injection de défauts . . . 13
9.2 Méthode : prédire puis mesurer . . . 14
9.3 Un bug trouvé puis corrigé : freinage parasite à 5 Hz . . . 14

10 ADAS Validation Studio (application web) 15
10.1 Architecture . . . 15
10.2 Fonctions . . . 15
10.3 Choix et incidents notables . . . 15

11 Agent : langage naturel → scénario d'essai 16
11.1 But et principe . . . 16
11.2 Schéma du scénario . . . 16
11.3 Convertisseurs comparés . . . 16
11.4 Jeu A (35 demandes) : résultats . . . 17
11.5 Jeu B (26 demandes) : mesure à prompt figé . . . 17
11.6 Intégration au Studio . . . 17

12 Tests automatiques 18

13 Registre des difficultés rencontrées 18

14 Limites et affirmations à ne pas faire **20**

15 Perspectives **21**

A Reproduire **21**

B Glossaire **21**

1 Contexte, objectifs et règles de travail

1.1 Objectif du projet

Le projet est un travail personnel mené pour se rapprocher du domaine automobile embarqué (ADAS, perception, validation). Il est parti d'une question de perception — *que gagne-t-on à fusionner un LiDAR et une caméra ?* — puis a été étendu en trois étapes successives, chacune construite sur la précédente :

1. une chaîne de perception caméra + LiDAR (détection, distance, rejet de fausses alarmes) ;
2. un AEB fondé sur cette perception, évalué en boucle fermée dans le simulateur CARLA ;
3. une application web (*ADAS Validation Studio*) pour lancer et analyser des essais et des campagnes, et un agent LLM qui traduit une demande textuelle (« voiture à l'arrêt à 50 m, j'arrive à 70 km/h ») en scénario valide.

1.2 Règles de méthode (cahier des charges de conduite du projet)

Ces règles ont guidé chaque décision et expliquent le ton prudent du rapport :

- **Vérifier avant de supposer** : lire le code et les sorties réelles plutôt que d'affirmer de mémoire.
- **Ne jamais inventer de résultat** : tout chiffre cité provient d'une sortie de commande ou d'un fichier de `reference/`.
- **Distinguer implémenté** (le code existe), **testé** (des tests automatiques passent), **validé en simulation** (mesuré dans CARLA dans des conditions précisées) et **planifié** (pas encore fait).
- **Expliquer les conséquences** d'un choix (ce qu'il améliore, ce qu'il dégrade).
- **Dire les limites d'accès** : l'assistant n'a pas de terminal sur la machine ; les commandes sont lancées par l'auteur, qui colle les sorties, et fait lui-même les *commits* Git.

1.3 Environnement

Système	Windows ; GPU NVIDIA RTX 4050 (6 Go)
Simulateur	CARLA 0.9.16 (mode synchrone, pas de 0,05 s = 20 Hz), serveur local `localhost:2000`
Python	3.12, environnement virtuel `venv` ; `torch 2.11.0+cu128`, `ultralytics 8.4.155`, NumPy, OpenCV
Application	FastAPI + Uvicorn ; page HTML/JavaScript sans framework (le choix React/-TypeScript initialement envisagé est reporté)
Tests	`pytest`, sans CARLA
Dépôt	`C:\Projets\carla-lidar-camera-fusion` (Git ; données `data/` et `runs/` non versionnées)

2 Vue d'ensemble et structure du projet

2.1 Chaîne de traitement

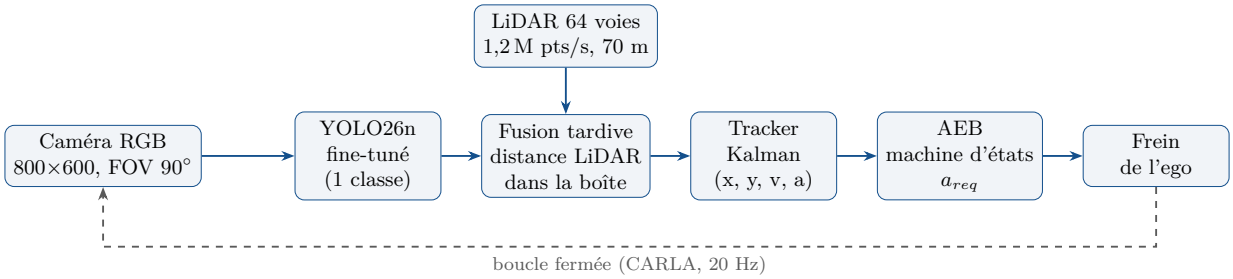


Figure 1: Chaîne de perception et de décision. En boucle fermée, le frein appliqué change ce que les capteurs voient à l'image suivante.

2.2 Arborescence

```
carla-lidar-camera-fusion/
  config / requirements.txt, README.md
  scripts/        00_rig_test ... 13_agent_eval        (une etape = un script)
  src/fusion/
    config.py      parametres du banc (capteurs, episodes, split)
    geometry.py    reperes, projection camera, transformations
    rig.py         capteurs CARLA, meteo, synchronisation, etiquetage automatique
    perception/    fusion camera-LiDAR (sans import carla : testable hors simulation)
    adas/          tracker de Kalman, AEB, banc cinematique, injection de defauts
    agent/         schema JSON, validation, prompt, client LLM, regles, metriques
  backend/
    app/main.py    API FastAPI (runs, campagnes, agent)
    worker.py      worker d'observation (YOLO + LiDAR) en sous-processus
    static/index.html   interface web
  reference/       resultats figes (CSV, jsonl, hash du modele)
  tests/           test_aeb, test_faults, test_perception, test_fusion_lateral, test_api,
    test_agent
  data/ runs/ models/  (non versionnes)
```

2.3 Scripts, du plus ancien au plus récent

Script	Rôle
`00_rig_test`	test du banc de capteurs
`01_collect`	collecte (épisodes × météos clair/pluie/brouillard), reprise possible après arrêt
`02_view, 03_audit`	visualisation et audit du jeu de données
`04_make_yolo`	conversion au format YOLO (une classe *vehicle*)
`05_eval_yolo`	évaluation du détecteur par distance et par météo (avant fusion)
`06_train_yolo`	entraînement
`07_leak_check, 08_novelty_recall`	contrôle de fuite train/test et recall selon la nouveauté de la vue

Script	Rôle
`09_fusion_depth`	fusion LiDAR : profondeur, comparaison à deux baselines caméra, rejet de fausses alarmes
`10_aeb_sweep`	balayage des seuils AEB sur un modèle cinématique (sans CARLA)
`11_aeb_closed_loop`	un essai AEB en boucle fermée dans CARLA
`12_aeb_campaign`	campagne : vitesses × météos × graines, tableaux agrégés
`13_agent_eval`	évalue la conversion langage naturel → JSON (sans CARLA)

3 Banc de simulation et collecte de données

3.1 Capteurs et paramètres

Les paramètres sont centralisés dans `src/fusion/config.py` (classe immuable `Config`), ce qui permet de retrouver exactement les conditions d'une collecte.

Paramètre	Valeur
Pas de simulation	0,05 s (20 Hz), mode synchrone
Sauvegarde	1 image sur 4 (5 Hz)
Ville de collecte	Town10HD_Opt
Épisodes	60, 60 images cibles chacun (12 s de conduite), 70 véhicules de trafic
Météos	`clear` (ClearNoon), `rain` (HardRainNoon), `fog` (nuages 80, brouillard 60, distance 20 m)
Caméra RGB	800×600, FOV 90°, position (1,5 ; 0 ; 2,4) m
LiDAR	64 voies, $1{,}2 \cdot 10^6$ pts/s, portée 70 m, FOV $+5°/-20°$, `dropoff_general_rate`=0
Étiquetage	distance maximale 60 m ; visibilité minimale 0,3
Découpage	par épisode : 70 % entraînement, 15 % validation, 15 % test

Choix : découpage par épisode, pas par image

Deux images successives d'un même épisode sont presque identiques. Un découpage aléatoire par image mettrait des quasi-doublons en entraînement et en test et gonflerait artificiellement les scores. Chaque épisode entier est donc affecté à un seul ensemble, et les trois météos d'un même épisode partagent les mêmes graines (mêmes trajectoires) pour comparer proprement les conditions. Avec 60 épisodes cela donne 9 épisodes de test (confirmé par les fichiers d'évaluation).

Choix : LiDAR sans suppression de points

CARLA supprime 45 % des points par défaut (`dropoff_general_rate`). Pour mesurer la fusion sans ce bruit artificiel non représentatif d'un capteur réel particulier, le taux est mis à 0. Conséquence : le LiDAR simulé est plus favorable que du réel, et il n'est pas dégradé par la pluie ni le brouillard (voir limites).

3.2 Étiquetage automatique

`rig.label_objects` produit pour chaque véhicule dans le champ :

- la boîte 2D serrée, obtenue par la **segmentation d'instance** de CARLA (pas par projection de boîtes 3D, qui sur-estime les véhicules partiellement cachés) ;
- la **visibilité** (pixels visibles / surface de l'enveloppe projetée) ; en dessous de 0,3 le véhicule est marqué « ignoré » ;
- les profondeurs de référence : face avant la plus proche (`depth_face`) et centre ;
- le nombre de points LiDAR tombant sur l'instance ;
- le nombre de véhicules dans le champ mais cachés (comptés, non étiquetés).

Choix : zone « ignorée » plutôt que faux négatif

Un véhicule presque totalement masqué n'est ni un succès ni un échec du détecteur. En entraînement, son emplacement est grisé dans l'image (les pixels des véhicules gardés sont restaurés par-dessus) pour ne pas apprendre à l'ignorer à tort ; en évaluation, une détection sur un véhicule ignoré n'est comptée ni comme réussite ni comme fausse alarme (voir `05_eval_yolo.py`).

3.3 Contrôles de qualité du jeu de données

- `03_audit` : audit général du jeu de données ;
- `07_leak_check` : pour chaque image de test, distance à l'image d'entraînement la plus proche de même cap ($\pm 30°$) ;
- `08_novelty_recall` : recall du détecteur selon cette distance (familier < 5 m, intermédiaire, nouveau ≥ 20 m).

Ces scripts existent et ont été utilisés, mais leurs sorties chiffrées ne sont pas rejouées dans ce rapport : elles n'ont pas été conservées dans `reference/`. Je ne cite donc pas de pourcentage de fuite.

4 Détecteur YOLO

4.1 Choix et entraînement

- Modèle de départ : `yolo26n.pt` (le plus petit, adapté à un GPU de 6 Go et à une inférence en boucle fermée à 20 Hz).
- Une seule classe *vehicle* : la perception sert à la distance d'un obstacle, pas à la catégorie.
- Paramètres (`reference/train_v4_n_args.yaml`) : 40 époques maximum, patience 10, lot de 8, images 800 px, optimiseur automatique, mosaïque, retournement horizontal, HSV, graine 0, déterministe.
- L'entraînement s'est arrêté à l'époque 38 (patience), meilleur modèle à l'époque 28, soit environ 76 min.

Meilleure époque sur la validation : mAP50 = 0,948 ; mAP50–95 = 0,771 ; précision 0,908 ; recall 0,900. L'empreinte SHA-256 du fichier de poids est conservée dans `reference/best_pt.sha256` pour retrouver exactement le modèle utilisé.

4.2 Résultats sur le jeu de test (recall par distance et météo)

Recall = part des véhicules étiquetés (non ignorés) retrouvés par une détection, par tranche de distance. Les chiffres ci-dessous sont recalculés depuis les CSV de `reference/`.

Modèle, ville	Météo	0–20 m	20–40 m	40–60 m
YOLO26n pré-entraîné (COCO)*, Town10	clair	99,6 %	91,9 %	69,7 %
	pluie	98,5 %	78,5 %	44,0 %
	brouillard	96,4 %	48,2 %	3,4 %
Fine-tuné, Town10 (4 451 objets, 9 ép.)	clair	100 %	98,8 %	89,3 %
	pluie	100 %	99,0 %	85,5 %
	brouillard	100 %	99,0 %	89,7 %
Pré-entraîné (COCO), Town03 (3 393 objets, 10 ép.)	clair	98,2 %	87,3 %	76,4 %
	pluie	97,2 %	74,3 %	60,3 %
	brouillard	97,5 %	62,0 %	6,9 %
Fine-tuné, Town03	clair	100 %	96,8 %	86,7 %
	pluie	100 %	95,9 %	82,0 %
	brouillard	100 %	95,5 %	89,7 %

* d'après le nom du fichier (`eval_yolo26n_test.csv`) et l'usage documenté de `05_eval_yolo.py -coco` ; classes COCO voiture, bus, camion.

Lecture honnête

Le fine-tuning résout surtout le brouillard à moyenne et longue distance (de 3,4 % à 89,7 % à 40–60 m). Mais le détecteur est entraîné sur *CARLA* Town10 : ces scores mesurent un détecteur de rendu synthétique sur des modèles de véhicules identiques à l'entraînement, pas une performance sur des images réelles. Town03 est une seconde ville, mais l'absence de fuite d'entraînement sur ses données reste à confirmer d'après l'organisation des dossiers (`data\v4_town03`) avant toute revendication.

5 Fusion LiDAR–caméra : distance et rejet de fausses alarmes

5.1 Méthode (fusion tardive)

La fusion est **tardive** (au niveau des objets détectés) : le détecteur 2D produit des boîtes, puis le LiDAR fournit la distance.

1. Les points LiDAR sont transformés dans le repère caméra, puis projetés dans l'image (`project_lidar`).
2. Les points du **sol** sont retirés : seuls ceux situés à plus de 0,30 m au-dessus du sol (`GROUND_MARGIN`) sont gardés.
3. On ne garde que les points dans la **zone centrale** de la boîte (60 % de sa largeur et de sa hauteur, `INNER`).
4. La distance est celle du **premier groupe** de profondeur (le plus proche) : c'est la face avant de l'obstacle, ce qui compte pour freiner.
5. Sans point LiDAR : pas de mesure (utile pour le rejet, et pour l'AEB : *pas de profondeur = pas de piste*).

Choix : fusion tardive

Elle exploite un détecteur 2D standard, garde chaque capteur diagnostiquable séparément (on peut mesurer ce que chacun apporte) et coûte peu de calcul. Une fusion précoce (nuage de points coloré) ou au niveau des caractéristiques demanderait un réseau dédié : hors de portée du GPU et du temps disponibles, et plus difficile à valider en boucle fermée.

Choix : premier groupe plutôt que médiane

La médiane des points de la boîte mélange l'obstacle et l'arrière-plan visible entre les roues ou au-dessus du toit. Le premier groupe donne la face avant, qui est la distance de freinage pertinente.

5.2 Comparaison à deux estimations caméra seule

Référence : face avant du véhicule (étiquette CARLA). Erreur = estimé − référence. Deux baselines caméra : (i) **plan du sol** (la distance se déduit du bas de la boîte, `ground_depth` $= f_y\, h_{cam}/(y_{bas} - v_0)$) ; (ii) **hauteur fixe** 1,5 m. Town10HD_Opt : 4 256 détections justes ; Town03 : 3 172.

Ville, distance	LiDAR : MAE (part < 1 m)	Plan du sol : MAE (part < 1 m)	Hauteur fixe 1,5 m
Town10, 0–20 m	0,83 (74 %)	1,01 (73 %)	0,90 (69 %)
Town10, 20–40 m	1,20 (75 %)	0,99 (57 %)	3,36 (31 %)
Town10, 40–60 m	1,67 (78 %)	1,29 (50 %)	6,27 (13 %)
Town03, 0–20 m	0,71 (77 %)	0,98 (73 %)	0,77 (73 %)
Town03, 20–40 m	1,10 (81 %)	2,32 (59 %)	3,46 (21 %)
Town03, 40–60 m	0,68 (93 %)	7,10 (33 %)	5,43 (12 %)

Lecture honnête

Le LiDAR n'est *pas* plus précis en moyenne partout : à Town10, le plan du sol a une MAE plus faible à 20–60 m (1,0 et 1,3 m contre 1,2 et 1,7 m), mais seulement 50–57 % des détections à moins de 1 m contre 75–78 %. À Town03, le plan du sol se dégrade nettement (MAE de 7,1 m à 40–60 m ; cause non vérifiée, probablement la pente de la route qui viole l'hypothèse de route plane). Le LiDAR reste stable d'une ville à l'autre (MAE de 0,7 à 1,7 m, couverture de 99 à 100 %, biais médian de +0,2 à +0,6 m, corrigé du pare-chocs dans l'AEB). La hauteur fixe est mauvaise au-delà de 20 m. **Il ne faut donc pas affirmer que le LiDAR bat la baseline plan-du-sol en précision à Town10** ; l'avantage est la robustesse (d'une ville à l'autre, et part de détections < 1 m).

5.3 Rejet des détections sans preuve LiDAR

Une détection n'est conservée que si au moins N points LiDAR tombent dans sa boîte.

	Town10		Town03	
	précision	recall	précision	recall
Sans rejet	92,1 %	95,6 %	86,9 %	93,5 %
$N = 3$	92,6 %	95,5 %	90,2 %	92,9 %
$N = 10$	94,6 %	86,1 %	94,0 %	79,1 %

Le rejet à $N = 3$ retire 7 % des fausses alarmes à Town10 et 28 % à Town03. L'effet est modeste ; $N = 10$ coûte trop de recall. Ces fausses alarmes sont celles du détecteur sur des images CARLA,

pas une mesure sur du trafic réel. Le seuil retenu pour l'AEB exige simplement une mesure LiDAR (le tracker n'accepte pas de détection sans profondeur).

5.4 Mesure latérale

Pour décider si une cible est dans le couloir de trajectoire, on a besoin de sa position latérale. Elle est calculée à partir des points LiDAR (option `-lateral-source lidar`, défaut) plutôt qu'à partir du centre de la boîte caméra. **testé**, mais cette mesure *n'a pas résolu seule* le freinage parasite à 5 Hz (voir section 9.3).

6 Suivi de cibles (filtre de Kalman)

Le tracker travaille dans le repère du véhicule (x devant, y latéral, droite positive). Les vitesses sont **relatives** à l'ego : un objet immobile devant un ego à 14 m/s a $v_x = -14$ m/s.

- **Modèle `cv`** : état (x, y, v_x, v_y), vitesse constante (référence historique).
- **Modèle `ca`** (défaut) : état $(x, y, v_x, v_y, a_x, a_y)$, accélération constante avec bruit de saccade (*jerk*, 2 m/s^3 dans le script 11).
- Mesure : distance de la face avant par LiDAR, écart-type de position 0,3 m ; association par porte $(2 + 25\,\Delta t$ m) ; piste confirmée après plusieurs mesures ; supprimée après 3 pertes consécutives.
- Initialisation : écart-type de vitesse longitudinale 8 m/s (une piste neuve ignore le rapprochement), mais **vitesse latérale 2 m/s** (voir §9.3).

Choix : modèle à accélération constante

Avec un modèle à vitesse constante, une cible qui freine est vue comme « encore rapide » pendant plusieurs images. L'accélération relative estimée permet d'*anticiper* l'arrêt de la cible (voir AEB). Prix : une estimation plus bruitée, donc un seuil minimal de décélération pour ne pas réagir au bruit.

`Track.misses` compte les images consécutives sans mesure ; il sert à la garde contre les pistes en roue libre (section suivante).

7 AEB : méthode et choix de conception

7.1 Machine d'états

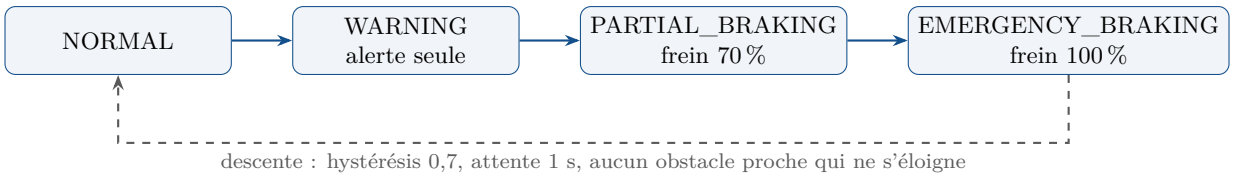


Figure 2: États de l'AEB. L'escalade exige la confirmation du niveau sur plusieurs images ; la descente est volontairement lente.

7.2 Critère principal : la décélération requise

Pour chaque cible dans le couloir ($|y| < 1{,}5$ m, distance < 60 m, rapprochement $> 0{,}5$ m/s), on calcule la décélération nécessaire pour s'arrêter avant l'obstacle, latence comprise :

$$a_{req} = \frac{v_c^2}{2\,(x - v_c\,t_{lat} - m)}, \qquad t_{lat} = 0{,}4 \text{ s},\ m = 1 \text{ m},$$

infinie si le dénominateur est négatif ou nul. v_c est la vitesse de rapprochement, x la distance. Niveaux : alerte si $a_{req} \geq 2{,}0$ m/s^2 ou TTC $< 2{,}6$ s ; freinage partiel (frein 70 %) si $a_{req} \geq 3{,}0$; urgence (frein 100 %) si $a_{req} \geq 4{,}0$.

Choix : a_{req} plutôt qu'un seuil de TTC fixe

Un TTC fixe donne la même alerte à 30 et à 70 km/h alors que la distance d'arrêt varie comme v^2 : un essai a montré qu'un seuil de TTC freine trop tard à 50 km/h avec un délai d'actionnement de 0,2 s. La décélération requise tient compte de la vitesse, de la distance et de la latence. Le TTC ne sert qu'à l'alerte.

Choix : seuils calibrés sur la voiture CARLA, pas sur une norme

La première version supposait 8 m/s^2 de décélération maximale. La mesure donne environ 4,1 m/s^2 à 70 km/h (frein 100 %) pour le véhicule de test. Les seuils (2,0 / 3,0 / 4,0) sont des valeurs de départ choisies après balayage en boucle fermée (jeu « B ») ; ce ne sont pas des valeurs homologuées. Le jeu précédent est gardé (`AEBConfig.conservative()` : 1,5 / 2,5 / 3,5, frein partiel 60 %) ; il freine environ 0,7 s plus tôt à 50 km/h mais déclenche à TTC ≈ 3 s, plus difficile à juger comme nécessaire.

7.3 Mesures de robustesse

1. **Confirmation** : le niveau doit être demandé sur 2 images consécutives de l'AEB, et la piste doit persister 0,15 s (4 mesures à 20 Hz, 3 à 10 Hz, 2 à 5 Hz) : filtre les fausses détections courtes.
2. **Hystérésis** : un niveau n'est quitté que si a_{req} tombe sous 0,7× son seuil. Sans cela, un freinage qui réussit fait osciller a_{req} autour du seuil, le frein est relâché à 16 m d'un obstacle fixe, et il faut ensuite freiner plus fort (observé en CARLA à 50 km/h).
3. **Maintien** : on ne relâche pas tant qu'un obstacle du couloir est à moins de 15 m sans s'éloigner ; un essai a montré qu'un relâchement à 1 m/s avec un conducteur qui ré-accélère ramène l'ego à 0,5 m de l'obstacle.
4. **Anticipation de la décélération de la cible** (`-lead-decel`, activée dans le script 11 avec le tracker `ca`) : si l'accélération absolue estimée de la cible est < -3 m/s^2 sur 3 images, on suppose qu'elle s'arrête et on calcule la décélération pour s'arrêter avant son point d'arrêt (plafond 8 m/s^2).
5. **Latence tenant compte de la fréquence de perception** : quand la perception tourne à moins de 20 Hz, l'AEB ajoute environ 3 périodes de calcul à la latence (maintien du frein + confirmation) ; c'est une hypothèse de modèle.
6. **Garde contre les pistes en roue libre** (`escalate_max_misses`=2) : une piste sans mesure depuis plus de 2 images est extrapolée, non observée ; elle ne peut plus faire *monter* le niveau, mais retient un frein déjà engagé. Une tolérance de 0 a été écartée : jusqu'à +9 collisions sur 40 à 5 Hz avec 30 % de pertes (banc cinématique).

8 Validation en boucle fermée dans CARLA

8.1 Banc de test

Le script `11_aeb_closed_loop` lance un scénario avec seulement l'ego et un véhicule cible (**sans trafic**). À chaque tick de 20 Hz : capteurs → YOLO → LiDAR → tracker → AEB → commande de l'ego. Le conducteur et le régulateur de l'ego sont écrits pour l'essai. Sorties par run : `log.csv` (une ligne par tick), `summary.json`, images annotées, `status.json`.

Scénario	Description et attente
`stopped`	cible à l'arrêt dans la voie ; l'AEB doit éviter la collision
`braking`	cible qui roule puis freine à $t =$ `-lead-brake-t` (frein 0,75 par défaut)
`adjacent`	véhicule à l'arrêt en voie voisine (décalage 3,5 m) : l'ego doit passer **sans freiner**
`follow`	cible à vitesse constante, sans freiner : l'ego ne doit **pas** freiner
`approach`	approche avec un régulateur de distance d'un véhicule plus lent : conduite normale, pas de freinage

Les scénarios *adjacent*, *follow* et *approach* mesurent les **freinages parasites** ; les deux premiers mesurent l'**efficacité**. Les campagnes (`12_aeb_campaign`) répètent un cas sur des vitesses, des météos et des graines ; à chaque essai un processus CARLA distinct est lancé (≈ 15 s).

8.2 Résultats, Town10HD_Opt, 20 Hz, météo claire

Scénario	Résultat
Cible arrêtée, 30 / 50 / 70 km/h (3 graines, écart de départ 50 m)	0 collision ; écart minimal médian 9,9 / 7,6 / 3,3 m ; TTC au premier freinage 1,8 / 2,5 / 2,4 s
Idem avec le jeu conservateur	0 collision ; 11,9 / 9,2 / 4,3 m ; TTC 2,1 / 3,2 / 2,4 s
Cible qui freine (1 s après le départ), 50 km/h, écart 20 m (2 graines)	0 collision, écart minimal 8,6 m (*sans* anticipation : 2/2 collisions, impact 23 km/h)
70 km/h avec plus de place (départ 60 m, 3 graines)	0 collision, écart minimal médian 13,2 m
Cible hors couloir, suivi à même vitesse, approche normale	0 freinage parasite observé (alerte seule en approche à 20 et 18 m)

Lecture honnête

À 70 km/h avec 50 m de départ, la distance est déjà inférieure à la distance d'arrêt : le freinage part dès la première détection quel que soit le seuil. Cette ligne ne départage donc pas les jeux de seuils. Les essais ont 2 à 4 graines : ils montrent un comportement, pas une distribution statistique.

8.3 Effet de la fréquence de perception

Cible arrêtée, départ 60 m, 2 graines, écart minimal médian :

Fréquence	20 Hz	10 Hz	5 Hz
50 km/h	7,6 m (départ 50 m)	8,0 m	5,3 m
70 km/h	13,2 m	11,5 m	6,5 m

Sans correction de la latence supposée, à 5 Hz le même jeu provoquait une collision (13 km/h) sur 1 essai sur 2 à 50 km/h. Les effets de cette correction sont validés sur la cible arrêtée seulement, pas sur les scénarios non dangereux à 5 et 10 Hz.

8.4 Seconde ville : Town03

Mêmes scripts et réglages, 2 graines (42 et 43), CARLA lancé **sans** l'option `-quality-level=Low`.

Cause non élucidée

Avec `-quality-level=Low`, le chargement de Town03 plante (« Shader compilation failures are Fatal »). Le rendu par défaut déplace les écarts à 50 km/h de moins d'un mètre : le témoin Town10 a donc été refait dans le même mode, et on ne mélange pas deux qualités dans une comparaison.

Scénario (écart 50 m sauf mention)	Town03, défaut	Town10, même rendu	Town10, Low
Cible arrêtée 30/50/70 : collisions	0/2 chacun	0/2 chacun	0/2 chacun
idem : écart minimal médian (m)	9,9 / 9,6 / 3,9	10,2 / 8,4 / 3,3	9,9 / 7,7 / 3,3
Cible qui freine (même vitesse), écart 20 m, 50 km/h	0/2, 8,5 m	0/2, 8,6 m	0/2, 8,6 m
Cible qui freine (30 km/h), écart 45 m, 50 km/h	0/2, 7,5 m (pire 6,7)	0/2, 8,0 m	non mesuré
Hors couloir 30/50/70, 20 et 5 Hz	0 freinage, 0 alerte (3 graines)	non refait	0 freinage (6 graines, 5 Hz)
Suivi même vitesse (15 m), 30/50 km/h	0/4 freinages	non refait	0/4

Les écarts entre villes sont du même ordre ($\leq$ 1,2 m pour des écarts de 7 à 10 m, avec 2 graines) : ce sont des indications, pas une différence mesurée.

8.5 Un cas non évitable, volontairement exclu des tableaux

Une cible à 30 km/h qui s'arrête en 0,5 s avec 20 m d'écart initial (ego à 50 km/h) donne 2 collisions sur 2 (impact $\approx$ 29 km/h) à Town03 comme à Town10. Le journal montre que l'alerte arrive à 0,2 s, mais que l'arrêt de la cible survient à 12–13 m ; la décélération nécessaire ($\approx$ 4,8 m/s^2, calcul approché) dépasse celle de la voiture ($\approx$ 4,1 mesurée), même sans latence. Un AEB ne corrige pas une situation physiquement inatteignable. Une erreur de ma part, repérée ici : une variante (`-gap 20` avec une cible à 30 km/h) avait été proposée sans validation ; elle est remplacée par des variantes validées (même vitesse, écart 20 m ; cible à 30 km/h, écart 45 m).

9 Robustesse aux fausses détections

9.1 Injection de défauts

Le module `FaultInjector` permet de simuler : des fausses détections dans le couloir (probabilité par image, durée réglable), des pertes de mesure, un bruit gaussien sur la distance. C'est un **test de contrainte**, pas un taux réaliste : les statistiques de persistance des fausses détections du détecteur réel ne sont pas mesurées.

9.2 Méthode : prédire puis mesurer

Les fréquences d'essais avec au moins un freinage parasite ont d'abord été *prédites* par le modèle hors CARLA (`tests/test_faults.py`, 200 simulations), puis *mesurées* en boucle fermée (50 km/h, route dégagée, probabilité 0,10 par image, ≈ 8 événements par essai de 4 s, 6 graines).

Durée	Modèle hors CARLA	CARLA, confirm. 2 mesures	CARLA, confirm. 0,15 s
1 image	14 %	1/6	0/6
2 images	49 %	3/6	0/6
3 images	98,5 %	6/6	0/6

Avec la confirmation de 0,15 s, le modèle prévoit 0 %, 17 % et 20 % pour 1, 2 et 3 images : le résultat 0/6 est *compatible* avec lui, il ne le prouve pas. Une fausse détection de 4 images ou plus freine toujours (100 % prédit, non mesuré en CARLA). Coût de la confirmation : environ 0,7 m de retard à 50 km/h si l'obstacle apparaît à 20–25 m (modèle hors CARLA) ; pas de régression sur la cible arrêtée à 20 Hz (mêmes écarts minimaux et distances de premier freinage).

9.3 Un bug trouvé puis corrigé : freinage parasite à 5 Hz

Symptôme : à 5 Hz, en passant à côté d'une cible arrêtée dans la voie voisine (50 km/h, graine 43), un freinage d'urgence parasite apparaissait dans 4 lancements sur 6.
Hypothèse écartée : l'erreur de mesure latérale. Passer à une mesure latérale par points LiDAR n'a pas résolu le problème à lui seul.
Cause trouvée en rejouant le journal : une fausse piste née de deux détections proches recevait une vitesse latérale initiale d'environ −5 m/s (a priori trop large de 8 m/s) et dérivait dans le couloir pendant 3 images.
Correction : à l'initialisation, écart-type de la vitesse latérale ramené à 2 m/s (`vy_init_std`). Test de non-régression `test_depassement_a_5hz_ne_declenche_pas_le_freinage`.
Validation : 0 freinage sur 8 lancements de la graine 43, et 0/18 sur les graines 42 à 47 à 30 / 50 / 70 km/h.
Garde ajoutée ensuite (`escalate_max_misses`=2) : neutralise la séquence rejouée même avec l'ancien a priori. Revalidée en CARLA : cible arrêtée et véhicule qui freine inchangés, 0/9 freinage hors couloir à 5 Hz.

Choix : rejouer le journal avant de modifier

La cause n'était pas la première hypothèse. Rejouer les mesures enregistrées dans le tracker a permis de voir la vitesse initiale aberrante, au lieu d'ajuster des seuils à l'aveugle (ce qui aurait masqué le défaut en dégradant l'efficacité).

10 ADAS Validation Studio (application web)

10.1 Architecture

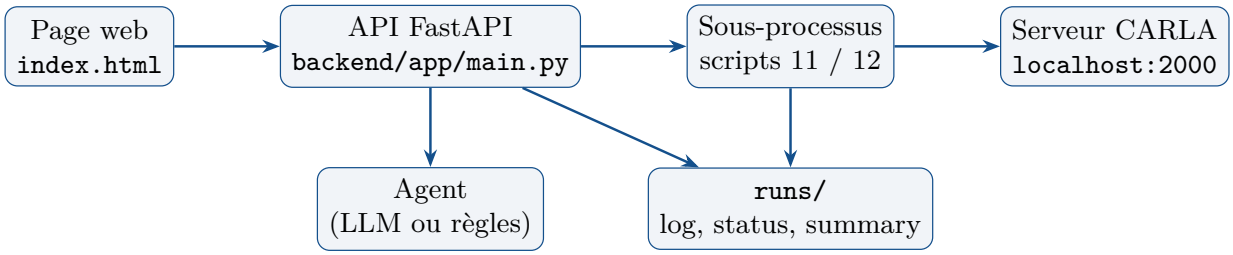


Figure 3: Le serveur ne contient pas de logique de simulation : il lance les scripts en sous-processus et lit les fichiers qu'ils écrivent.

Choix : sous-processus plutôt que simulation dans le serveur

CARLA, YOLO et le GPU sont des ressources lourdes et fragiles ; un plantage de CARLA ne doit pas tuer l'API. Un processus par essai donne une isolation naturelle, un arrêt propre (fichier `STOP`), et réutilise exactement les scripts validés en ligne de commande. Contrepartie : environ 15 s d'initialisation par essai, et un seul run ou une seule campagne à la fois (une instance CARLA, un GPU).

10.2 Fonctions

- **Run d'observation** : trafic, YOLO + distances LiDAR en direct.
- **Run AEB** en boucle fermée (script 11) : vue vidéo en direct (images à 10 Hz avec petite mémoire tampon), état de l'AEB, décélération requise, écart ; bilan et courbes à la fin.
- Options : AEB désactivé (témoin), fréquence de perception 20/10/5 Hz, injection de défauts.
- **Campagnes** (`POST /api/campaigns`) : vitesses × météos × graines, maximum 60 essais, progression, tableau agrégé (collisions, impact moyen, écart médian et pire, écart et TTC au premier freinage), reprise sur le dossier de campagne, arrêt.
- **Valeurs par défaut par scénario** et indication de la **route droite nécessaire**, car le tronçon droit disponible est limité (95 m à Town10HD_Opt, 80 m à Town03).

10.3 Choix et incidents notables

- **Longueur de route** : une erreur « plus long tronçon droit : 95 m, il en faut 110 » a conduit à calculer la route requise par scénario (freinage : écart $+ 5 + v_l\, t_b + v_l^2/8 + 10$; suivi/approche : écart $+ 5 + 4v_l + 15$; adjacent et arrêt : écart $+ 30$), à l'afficher dans le formulaire et à refuser les cas infaisables avant de lancer CARLA.
- **Verrous de fichiers Windows** : le navigateur lisant `status.json` provoquait un `PermissionError` lors du renommage atomique. Solution : `replace_retry` (réessais de quelques ms) ; les écritures de fichiers d'affichage sont non critiques (on saute une mise à jour), celles de l'état final sont critiques.
- **Chargement de Town03** : délai de `load_world` porté à 180 s ; la ville passée en argument du serveur était ignorée, on la charge par l'API.

- **Test qui lançait un vrai processus** : un test de refus de campagne lançait une vraie campagne quand CARLA était allumé. Il remplace désormais l'état de CARLA et interdit `Popen`.

Statut de l'interface

L'API des campagnes et la page sont **implémenté** et **testé** sans CARLA. Une campagne *lancée depuis la page* jusqu'à un tableau agrégé cohérent avec la ligne de commande (arrêtée 30/50/70, graines 42 et 43, attendu ≈ 10 / 7,6 / 3,3 m, 0 collision) **n'est pas encore confirmée par l'auteur** : elle n'est donc pas **validé en simulation**.

11 Agent : langage naturel → scénario d'essai

11.1 But et principe

Convertir une phrase comme « Une voiture à l'arrêt à 60 m, j'arrive à 50 km/h sous la pluie, essaie les graines 42 et 43 » en un scénario JSON que le Studio sait lancer, ou la **refuser avec une raison**. Le LLM ne lance rien : il écrit un JSON, qui est ensuite **validé par du code déterministe**.

11.2 Schéma du scénario

Champ	Domaine
`scenario`	stopped, braking, adjacent, follow, approach
`ego_speed_kmh`	[10, 90]
`gap_m`	[10, 80]
`lead_speed_kmh`	[0, 90] (≥ 5 pour braking, follow, approach)
`lead_brake_t_s`	[0, 20]
`weather`, `town`, `seeds`	clear/rain/fog ; villes du Studio ; liste de graines
`perception_hz`	20, 10 ou 5
`aeb`	booléen
`faults`	ghost_prob [0 ;0,2], ghost_len [1 ;30], drop_prob [0 ;0,9], noise_sigma [0 ;2]

Chaque scénario a des valeurs par défaut (par ex. `stopped` : écart 50 m ; `braking` : écart 45 m, cible 30 km/h, freinage à 1 s ; `follow` : écart 15 m, cible à la vitesse de l'ego).

Choix : trois niveaux de validation après le LLM

(1) **format** : champs connus, types, unités, plages ; (2) **faisabilité** : la route droite requise est comparée à la route disponible dans la ville choisie ; (3) **reconstruction** d'un corps de requête pour l'API (`to_studio_payload`). Un modèle peut se tromper de façon plausible ; la vérification doit donc être indépendante du modèle. Un refus est un résultat valide : *il vaut mieux refuser qu'exécuter un essai physiquement impossible.*

11.3 Convertisseurs comparés

- **Baseline à règles** (expressions régulières, sans LLM) : reproductible, sert de plancher.
- **LLM** via un client pour API « compatibles OpenAI » (bibliothèque standard uniquement ; Groq, OpenRouter, Ollama, Gemini ; réessais sur 429/5xx ; `User-Agent` explicite car un pare-feu renvoyait une erreur 403 « 1010 » sans en-tête). Température 0, suppression des blocs `<think>`.

- Deux prompts : v1 (886 caractères) et v2 (1 300 caractères, qui précise que la vitesse donnée est celle de l'ego).

Choix du fournisseur : une API gratuite (décision d'octobre 2026, après avoir envisagé d'abord le modèle le plus puissant). Les limites gratuites de Groq relevées (modèles `openai/gpt-oss-120b`, `qwen/qwen3.8-27b` ; 30 requêtes/min, 1 000/jour, 8 000 tokens/min, 200 000 tokens/jour) proviennent de pages d'agrégateurs datées de mi-octobre 2026, *pas d'une documentation officielle* : à revérifier.

11.4 Jeu A (35 demandes) : résultats

Jeu `reference/agent_benchmark.jsonl` : 35 demandes en français et en anglais, **dont 7 à refuser** (infaisables ou trop vagues), écrit par moi. Un cas est correct si tous les champs coincident (tolérance de 0,5 pour les valeurs ≥ 5) ou si le refus attendu est produit.

Convertisseur	Cas corrects sur 35
Règles (mots-clés)	23 (65,7 %), reproductible
`openai/gpt-oss-120b` (Groq), prompt v1	33, puis 32 (deux passages)
Idem, prompt v2	34, puis 33, 33, 32 (moyenne 32,7)

Lecture honnête

Sur les scénarios que le modèle accepte, les champs sont exacts (aucune erreur d'unité ni de valeur observée). Les échecs sont des **refus à tort** de demandes valides (4 cas selon le prompt) et, une fois, une demande vague acceptée. À température 0, deux passages identiques ne donnent pas le même score : une différence de 1 à 2 cas sur 35 n'est pas une amélioration. Le prompt v2 corrige deux cas que j'avais formulés de façon ambiguë et en casse deux de suivi : **aucun gain démontré** sur v1. Surtout, le prompt v2 a été ajusté d'après les erreurs de ce jeu : ce n'est pas une mesure sur données indépendantes.

11.5 Jeu B (26 demandes) : mesure à prompt figé

Pour obtenir une mesure plus honnête, un second jeu `reference/agent_benchmark_b.jsonl` a été écrit *après* le gel du prompt v2 : 26 demandes (catégories : familier, multi-paramètres, unités, défauts, anglais, infaisable, ambigu), dont 7 à refuser. Les réponses de référence sont validées par le code (aucune référence invalide ou infaisable) ; l'oracle obtient 26/26 et la baseline à règles 9/26 (35 %). **Le score du LLM sur ce jeu n'est pas encore mesuré** (commande préparée : `13_agent_eval.py -bench reference\agent_benchmark_b.jsonl -repeat 3`, sans retoucher le prompt). Limite persistante : ce jeu est encore écrit par moi ; un jeu C avec les formulations de l'auteur du projet est prévu.

11.6 Intégration au Studio

implémenté et **testé** sans CARLA ni vrai LLM (tests avec un faux modèle) :

- `GET /api/agent` : mode (`llm` ou `rules`), fournisseur, modèle. Le LLM est utilisé si la variable d'environnement `LLM_API_KEY` existe côté **serveur** (ou si le fournisseur est Ollama, local) ; sinon repli sur les règles, avec mention dans l'interface.
- `POST /api/agent/parse` : renvoie soit le scénario normalisé et la route nécessaire, soit un refus de type `rejet` (avec la raison donnée par le modèle), `invalide`, `infaisable`

ou `json_invalide`. Une erreur réseau ou de quota renvoie 502. **Ne lance jamais de processus.**

- Interface : zone de texte, bouton « Convertir », résultat lisible, bouton « Appliquer au formulaire de campagne » ; l'utilisateur relit puis lance lui-même.
- La campagne du Studio accepte désormais la carte (`town`), l'instant de freinage de la cible, la durée des fausses détections, les pertes et le bruit.

Choix : l'agent propose, l'humain décide

Un LLM peut se tromper de façon plausible. Le formulaire pré-rempli est **relu et lancé par l'utilisateur**. La clé d'API n'est jamais lue depuis le navigateur ni écrite dans un fichier du dépôt : elle reste dans la variable d'environnement du terminal. (Deux clés ont été collées par erreur dans la conversation : elles doivent être révoquées.)

Ce qui reste à faire sur l'agent

Pas de RAG, pas de LoRA, pas de boucle d'amélioration continue avec regroupement des erreurs en production ; un seul modèle testé ; l'agent connaît un seul interlocuteur (scénarios du Studio) ; le mode « sans AEB » d'une campagne n'est pas géré directement (le Studio lance l'AEB, et les témoins si la case est cochée). Ces éléments sont **planifié**.

12 Tests automatiques

64 tests passent hors CARLA sur ma copie de travail ; ils doivent être relancés par l'auteur sur sa machine (`python -m pytest -q tests`). Répartition par fichier :

Fichier	Contenu
`test_aeb.py`	machine d'états, décélération requise, hystérésis, anticipation, non-régression du freinage parasite à 5 Hz, garde en roue libre
`test_faults.py`	modèle hors CARLA des fausses détections (prédiction des fréquences de freinage parasite)
`test_perception.py`	fusion, `ground_depth`, rejet
`test_fusion_lateral.py`	mesure latérale par points LiDAR
`test_api.py`	runs, campagnes (commande construite, refus, suivi), agent (LLM simulé, refus, JSON illisible, infaisable, repli règles, aucun `Popen`)
`test_agent.py`	schéma, validation, faisabilité, métriques, baseline, client HTTP (faux serveur local), jeu B, prompts

Les tests vérifient le *code*, pas la physique : la physique se vérifie en CARLA (sections précédentes).

13 Registre des difficultés rencontrées

Problème	Cause	Solution / enseignement
Freinage parasite à 5 Hz (graine 43)	a priori de vitesse latérale initiale trop large (8 m/s)	écart-type 2 m/s ; garde `escalate_max_misses`=2 ; test de non-régression

Problème	Cause	Solution / enseignement
Garde de tolérance 0	les pistes de 1 image sans mesure sont légitimes à 5 Hz avec pertes	tolérance 2 ; 0 donnait +9 collisions sur 40
Fausses détections de 1–3 images	confirmation trop courte	confirmation 0,15 s ; modèle hors CARLA prédit puis CARLA mesure
Relâchement du frein trop tôt	a_{req} oscille autour du seuil	hystérésis 0,7 ; maintien proche d'un obstacle
Collision à 5 Hz sans correction	latence réelle supérieure à celle supposée	latence tenant compte de la fréquence ; validée sur cible arrêtée seulement
`PermissionError` sur `status.json`	lecteur (navigateur) ouvre le fichier pendant le renommage	`replace_retry` ; fichiers non critiques sautés
Route trop courte (110 m requis, 95 m dispos)	scénario choisi sans calcul préalable	formules de longueur, indication dans l'UI, refus avant lancement
Plantage de CARLA à Town03	`-quality-level=Low` : « shader compilation »	CARLA sans option ; cause non élucidée ; ne pas mélanger les qualités
Test qui lançait une vraie campagne	CARLA joignable lors des tests	état CARLA simulé, garde sur `Popen`
Cas « cible 30 km/h, écart 20 m »	physiquement inatteignable ($4{,}8 > 4{,}1$ m/s^2)	exclu des tableaux ; variantes validées
Erreur HTTP 403 « 1010 » (Groq)	absence de `User-Agent`	en-tête explicite ; test ajouté
Clés d'API collées dans le chat	erreur d'usage	révocation ; variable d'environnement du terminal uniquement
Fichiers non écrits sur le disque via le pont	écriture silencieuse sans effet	vérification systématique des tailles après écriture
Scores LLM non reproductibles	température 0 non déterministe côté API	répétitions, moyenne, pas de conclusion sur 1–2 cas

14 Limites et affirmations à ne pas faire

Limites générales

- **Simulation uniquement** ; deux villes ; 2 à 6 graines. Ces essais montrent un comportement ; ils ne constituent ni une validation statistique ni une homologation.
- Le LiDAR CARLA n'est pas dégradé par la pluie ni le brouillard : l'axe météo ne teste que la caméra.
- Modèles de véhicules identiques entre entraînement et test ; baseline caméra seule naïve (hauteur fixe 1,5 m) ; pas de trafic dense dans les essais AEB.
- Banc de test maison : conducteur et régulateur de l'ego écrits pour l'essai.
- L'AEB est validé à 20 Hz. À 5 Hz et 70 km/h, la marge est mince : écart minimal $\approx$ 2,4 m sur 16 lancements sans collision, mais **une collision** (impact 16 km/h) observée une fois sur la graine 42 dans un autre lot, non reproduite, cause non élucidée (variabilité du simulateur ou marge insuffisante).
- Fausse détection de 4 images ou plus : l'AEB freine toujours (non mesuré en CARLA).
- Les freinages parasites ne sont mesurés que sur trois scénarios simples.
- L'agent : jeu de test écrit par moi, prompt ajusté dessus, un seul modèle LLM.

Affirmations à ne pas faire (par exemple dans un CV ou un entretien) :

- « Le LiDAR est plus précis que la baseline caméra » (faux à Town10 en MAE moyenne).
- « 0 collision à 5 Hz et 70 km/h » (une collision observée).
- « L'AEB est validé / homologué » (calibré sur une voiture simulée, seuils de départ).
- « Le détecteur généralise » à des images réelles ou à une ville dont l'absence de fuite d'entraînement n'est pas confirmée.
- « L'agent LLM atteint X % » sans préciser qu'il s'agit d'un jeu écrit par moi, avec un prompt ajusté sur lui ; un score sur le jeu B ou un jeu C indépendant n'existe pas encore.

Formulations défendables : *« Pipeline LiDAR–caméra avec AEB validé en boucle fermée dans CARLA sur deux villes, avec analyse d'erreurs et corrections documentées »* ; *« Agent de conversion langage naturel → scénario JSON avec validation déterministe, évalué sur un jeu de demandes avec baseline à règles »*.

15 Perspectives

planifié	Contenu
Mesure du jeu B	lancer le LLM sur 26 demandes, prompt figé, 3 passages
Jeu C indépendant	phrases écrites par l'auteur, réponses attendues corrigées par lui
Comparaison Ollama local	modèle de 7–8 milliards de paramètres quantifié (faisable sur 6 Go de VRAM) : justifierait ou non un LoRA / un RAG
RAG et LoRA	seulement si le jeu indépendant montre une marge que le prompt ne comble pas
Amélioration continue	regroupement des erreurs de l'agent (catégories déjà produites par les métriques) puis correction mesurée
Validation de l'interface	campagne réelle lancée depuis la page, comparaison à la ligne de commande
Perception	mesure de la persistance réelle des fausses détections du détecteur ; trafic dense ; LiDAR dégradé par la météo
AEB	défense contre les fausses détections de 4 images ou plus ; marge à 5 Hz et 70 km/h

A Reproduire

```
python -m venv venv ; venv\Scripts\activate
pip install -r requirements.txt       # + wheel CARLA (non installable depuis PyPI)
# serveur CARLA lance (sans -quality-level=Low pour Town03)
python scripts\01_collect.py
python -m pytest -q tests             # sans CARLA
python scripts\12_aeb_campaign.py --scenario stopped --speeds 30 50 70 --weathers clear --seeds
    42 43 44 45
python -m uvicorn app.main:app --app-dir backend --port 8000     # Studio sur http://localhost
    :8000

# Agent (la cle reste dans le terminal, jamais dans un fichier du depot)
$env:LLM_API_KEY = Read-Host "cle"
$env:AGENT_PROVIDER = "groq"
python scripts\13_agent_eval.py --model openai --provider groq --llm openai/gpt-oss-120b `
   --sleep 6 --prompt v2 --bench reference\agent_benchmark_b.jsonl --repeat 3
```

Les résultats de référence sont dans `reference/` (CSV de fusion, d'évaluation du détecteur et des campagnes, empreinte du modèle, jeux de l'agent). Variables du serveur pour l'agent : `LLM_API_KEY`, `AGENT_PROVIDER`, `AGENT_MODEL`, `AGENT_BASE_URL`, `AGENT_PROMPT`.

B Glossaire

ADAS Advanced Driver Assistance Systems, aides à la conduite.

AEB Autonomous Emergency Braking, freinage d'urgence automatique.

TTC Time To Collision, temps avant collision $= x/v_c$.

a_{req} Décélération requise pour s'arrêter avant l'obstacle, latence comprise.

Boucle fermée La commande de l'ego modifie ce que les capteurs perçoivent ensuite (par opposition à un rejeu de données).

SIL	Software-In-the-Loop : le logiciel à tester tourne dans un simulateur.
Fusion tardive	Les capteurs sont combinés après la détection d'objets, pas au niveau des données brutes.
MAE	Erreur absolue moyenne.
Graine	Entier qui fixe les aléas (trajectoire, trafic) pour rendre un essai reproductible.
RAG / LoRA	Récupération de documents ajoutés au prompt / adaptation légère des poids d'un modèle ; **planifié** dans ce projet.